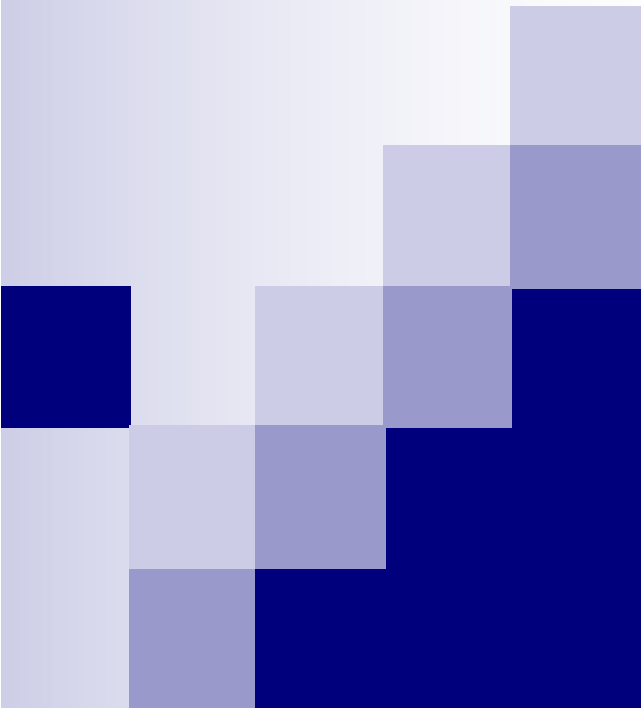




アルゴリズムとデータ 構造

第1回 ガイダンス
アルゴリズムとデータ構造とは？
最大公約数(GCD)のアルゴリズム

第1回ガイダンス

■ 今日の内容

- アルゴリズムとデータ構造とは何か
- 今日のアルゴリズム: 最大公約数問題を解く「ユークリッドの互除法」を学ぼう

■ ポイント

- アルゴリズムとデータ構造の定義
- 「ユークリッドの互除法」を理解すること
(情報理工学演習IIIを取っている人は初回の課題)



アルゴリズムはなぜ必要？

ビッグデータ, 人工知能, ..., アルゴリズム

ワトソン君

- IBMリサーチ (2011/02/16)
- クイズ番組で
- 100万冊の本
- 人工知能と自
ズム, 検索の技

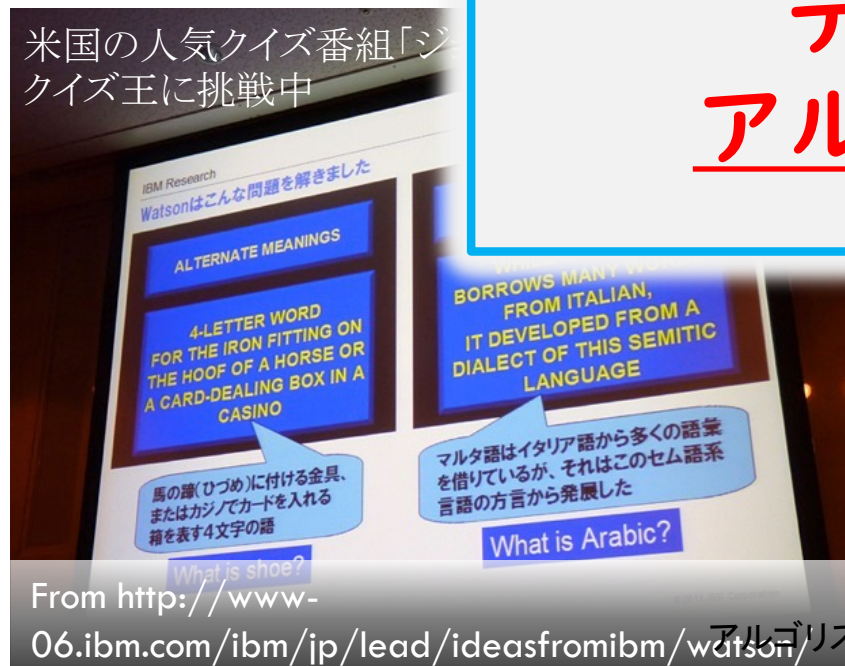
クラウド計算

世界最大の計算能力を計算!

AIの原動力は？

ハードウェア・
データ・
アルゴリズム

米国の人気クイズ番組「ジ
クイズ王に挑戦中



From [http://www-](http://www-06.ibm.com/ibm/jp/lead/ideasfromibm/watson/)

06.ibm.com/ibm/jp/lead/ideasfromibm/watson/ アルゴリズムとデ



From geek.com: Google server firm
<http://www.geek.com/articles/chips/up-next-for-google-enterprise-wars-2009078/>

プログラム言語の歴史

- 1940s-1950s 古代: 機械語、アセンブラ
- 1960s: FORTRAN, LISP, ALGOL (PASCAL)
- 1970s: C言語
- 1980s: SmallTalk
- 1990s: C++, Java, Objective C
- 2000s: Python, Ruby, Javascript...
- 2010s: Scala, Rust, Typescript, ...

プログラム言語の歴史

他のも
あるよ

- 1940s-1950s 古代: 機械語
- 1960s: FORTRAN, ALGOL, COBOL, PASCAL)
- 1970s: C言語
- 1980s: SmallTalk, Lisp, Prolog
- 1990s: C++, Java, Perl, JavaScript
- 2000s: Python, Ruby, PHP, Swift
- 2010s: Scala, Rust, TypeScript, Kotlin

答え

データ構造
入ってます!

クイズ: 新しいプログラム言語はどこが違うか? Intel™

なぜ「アルゴリズムとデータ構造」を学ぶのか？

効率的なプログラムを作成できる
ようになるため

効率的とは

- 計算時間が短い
- メモリ使用量が少ない

メリット

- アルゴリズムは**全てのプログラム言語**で役に立つ
- 情報理工学専攻大学院**入試に出題される科目**
- **IT関連に就職**する人には常識とされる



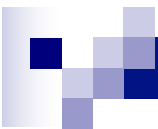
アルゴリズムって何？

アルゴリズムとデータ構造とは？

アルゴリズム

- ◆ 入力データから正しい出力を計算するために、一連の手順を記述したもの.
- ◆ 正しさと効率が重要.

||



計算問題とは

入力に対して、どのような出力が答えとなるかを書いたもの

(数学的にいうと、入力と出力の正しい組み合わせを何らかの方法で述べたもの)

アルゴリズムとは

計算問題を解くための
機械的に実行可能な手続き

(例) 自然数 a_0, a_1 ($a_0 \geq a_1$) の最大公約数を求める単純なアルゴリズム

「 n が a_0 と a_1 を共に割り切るかを、 $n = a_1$ から始めて1つずつ小さな値 n に対してチェックし、初めて割り切る値 n を出力」

計算問題の例

最大公約数問題 (GCD)

入力: 2個の正整数 a, b

出力: a と b の最大公約数.

最小全域木問題 (MST)

入力: 無向グラフ

$G=(V,E)$, 各枝の長さ

$Len: E \rightarrow R$.

出力: G の最小全域木.

部分集合和問題 (SUBSET SUM)

入力: $n+1$ 個の正整数

$a_1, \dots, a_n, b$

出力: 要素の和がちょうど b になるような $\{a_1, \dots, a_n\}$ の部分集合 S のどれか一つ

最短路問題

入力: 無向グラフ

$G=(V,E)$, 頂点 s, t .

出力: s から t への最短路のどれか一つ

「グラフ」というのは頂点が辺でつながった「ネットワーク」のこと. あとでやります.

アル・フワリズミ(紀元780-850)



古代バグダットの数学者 兼, 哲学者, 地理学者 (Muhammad ibn Mūsā al-Khwārizmī)

著書「アラビア数字の計算法」を執筆。「アルゴリズム」という用語は、アル・フワリズミの名前にちなんでつけられた。

「アルゴリズム」= ある数を求めるための定められた計算の手順

photo: <http://ja.wikipedia.org/wiki/%E0%B7%9C%E3%A1%8A%E3%81%A8%E3%83%BC%E3%83%AD%E3%83%BB>"Muhammad ibn Musa Al-hwarizmi"

アルゴリズムの表現

1. 自然言語の文章による表現

長: 特別に知識を必要としない、読みやすい

短: 厳密に書くのが難しい、
制御構造を表現しづらい

[日本語の文章による表現]

正整数 $n=a_1$ から始めて、 n が a_0 と a_1 を共に割り切るかを、1つずつ小さな値 n に対してチェックし、初めて割り切る値 n を出力せよ

2. プログラミング言語による表現

長: 厳密に書ける、制御構造を表現しやすい

短: 読みにくい、記述言語の知識が必要

```
for (n=a1; n>0; n--) {  
    if (a0%n==0 && a1%n==0)  
        return n;  
}
```

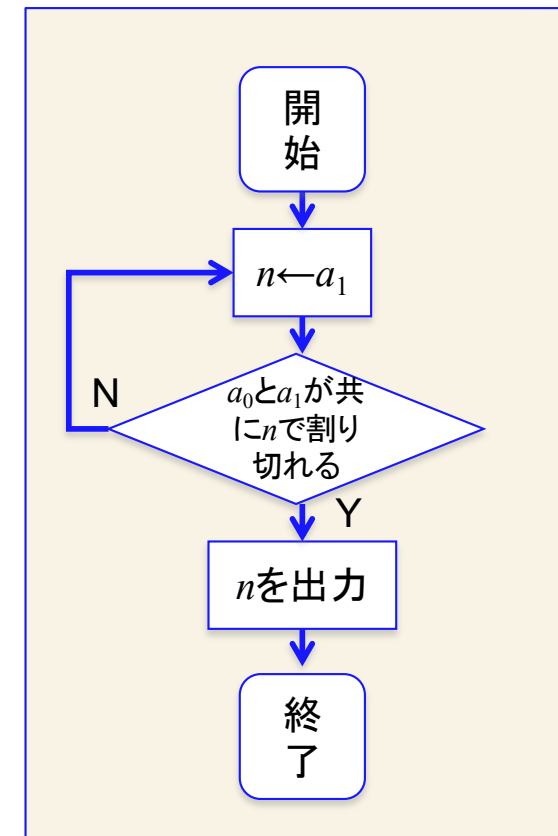
[C言語による表現]

3. フローチャートによる表現

長: 制御構造を理解し易い

短: 読みにくい

(最近はあまり使われない)



アルゴリズムの擬似コードによる表現

擬似コードによる表現

```
 $n \leftarrow 1$   
while (true) {  
  if  $a_0$ と $a_1$ が共に $n$ で割り切れる then  
    return  $n$   
  end if  
   $n \leftarrow n - 1$   
}
```

無限に繰り返す

n の値を返してこの
手続きから抜ける

長：読みやすい、制御構造を表現しやすい、
短：制御構造記述の知識が少し必要

制御構造の例

- 条件文 (if-then文, if-then-else文)
- くり返し文 (for文, while文)



データ構造って何？

アルゴリズムとデータ構造とは？

アルゴリズム

- ◆ 入力データから正しい出力を計算するために、一連の手順を記述したもの.
- ◆ 正しさと効率が重要.

データ構造

- ◆ 計算のために、記憶領域に効率良くデータを格納するための配置法.
- ◆ 時間と記憶領域を効率化.

授業で学ぶデータ構造の範囲

機械語のデータ型

- ◆ レジスタ値とその番地

基本データ型

- ◆ char, int, large int, double

構造データ型

- ◆ 配列 (array)

機械語
(型がない)

「プログラミング」
で学ぶ範囲

昔の言語も持つ型
(C, Pascal)

最近の言語が持つラ
イブラリ (C++, Java,
etc.)

「アルゴリズムと
データ構造」で
学ぶ範囲

基本的データ

- ◆ リスト (linked list), スタック (stack), 待ち行列 (queue)

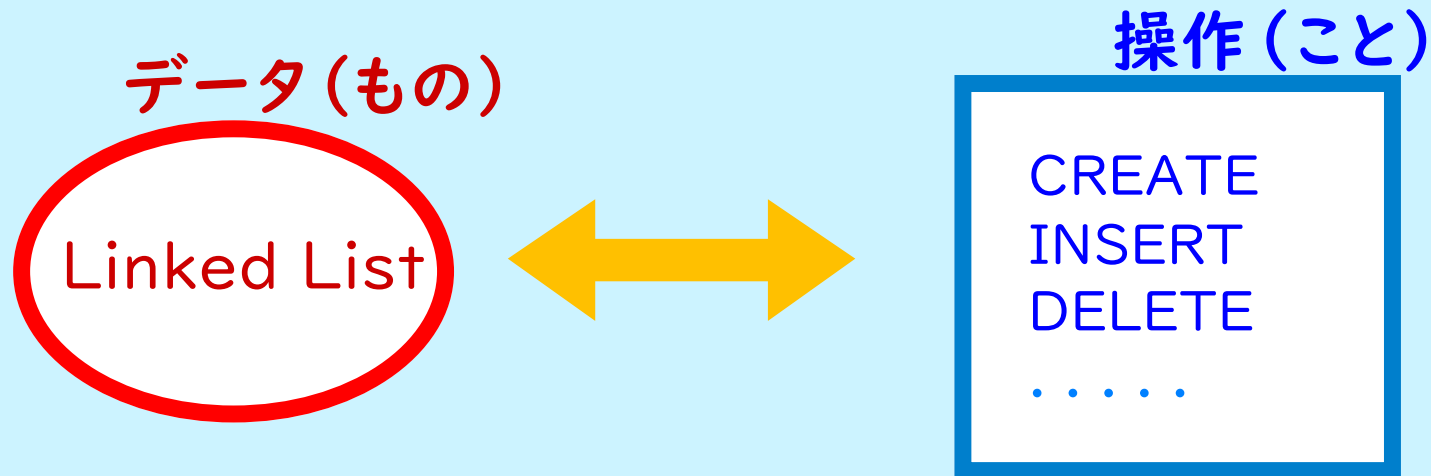
先進的データ構造

- ◆ 二分探索木 (binary search tree), 平衡探索木 (balanced search tree), ハッシュ表 (hash table)

大事な考え方: 抽象データ型

抽象データ型 (Abstract Data Type)

= データ型を, それに適用される一組の操作で
抽象的に定めたもの.



- データ構造には, 「どのように使うか?」と「それをどのように実現するか?」の二つの面がある.
- 抽象データ型の考え方は, とても重要である. 最近のほとんどのプログラム言語やライブラリーに取り入れられている. (例: C++, Java, Ruby, Python などなど)

データ構造を使うメリット

- データを**効率良く格納**できる

- 時間とメモリ



- 仕事が**抽象化**できる

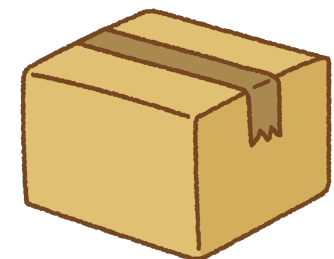
- 演算として考えられる



- **モジュール化**できる

- 仕様（仕事）と実装（プログラム）を分けて考えられる

- 誰もがつかえる（標準的ライブラリ）



みなさんはすでにデータ構造の考え方を分かっている

- ネットで, JavaのArrayList (C++のvector) の使い方を調べると...

docs.oracle.com

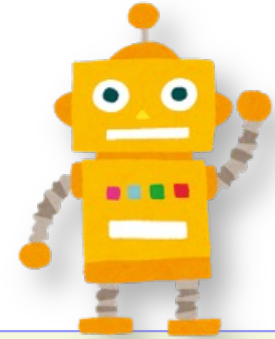
Jobs at Hokkaido University Google Google Scholar Remember The Milk Evernoteに保存 JSAI2016 HP

受信トレイ (12,408)... Google カレンダー... ☆総務成田さん：科... c++ - vectorとlist... ArrayList (Java Pl...

メソッドのサマリー

メソッド	メソッドと説明
boolean	add(E e) リストの最後に、指定された要素を追加します。
void	add(int index, E element) リスト内の指定された位置に指定された要素を挿入します。
boolean	addAll(Collection<? extends E> c) 指定されたコレクション内のすべての要素を、指定されたコレクションのイテレータによって返される順序でリストの最後に追加します。
boolean	addAll(int index, Collection<? extends E> c) 指定されたコレクション内のすべての要素を、リストの指定された位置に挿入します。
void	clear() リストからすべての要素を削除します。
Object	clone() この ArrayList インスタンスのシャローコピーを返します。
boolean	contains(Object o) 指定の要素がリストに含まれている場合に true を返します。
void	ensureCapacity(int minCapacity) この ArrayList インスタンスの容量を必要に応じて増やし、少なくとも最小容量引数で指定される要素数を保持できるようにします。

データ構造の勉強をしよう!



自分で一度、データ構造の仕組みを勉強してみると...

- (もちろん) **自分で作れる**ようになる

中身がわかる!

- C++やJava言語の**ライブラリ**が
何をしているかわかるようになる

「ブラックボックス」
じゃない

- **開発時間が短く**できる

サクサク書ける...

- 他の方の**プログラム**をつかえる/
他の方に**プログラム**をあげられる

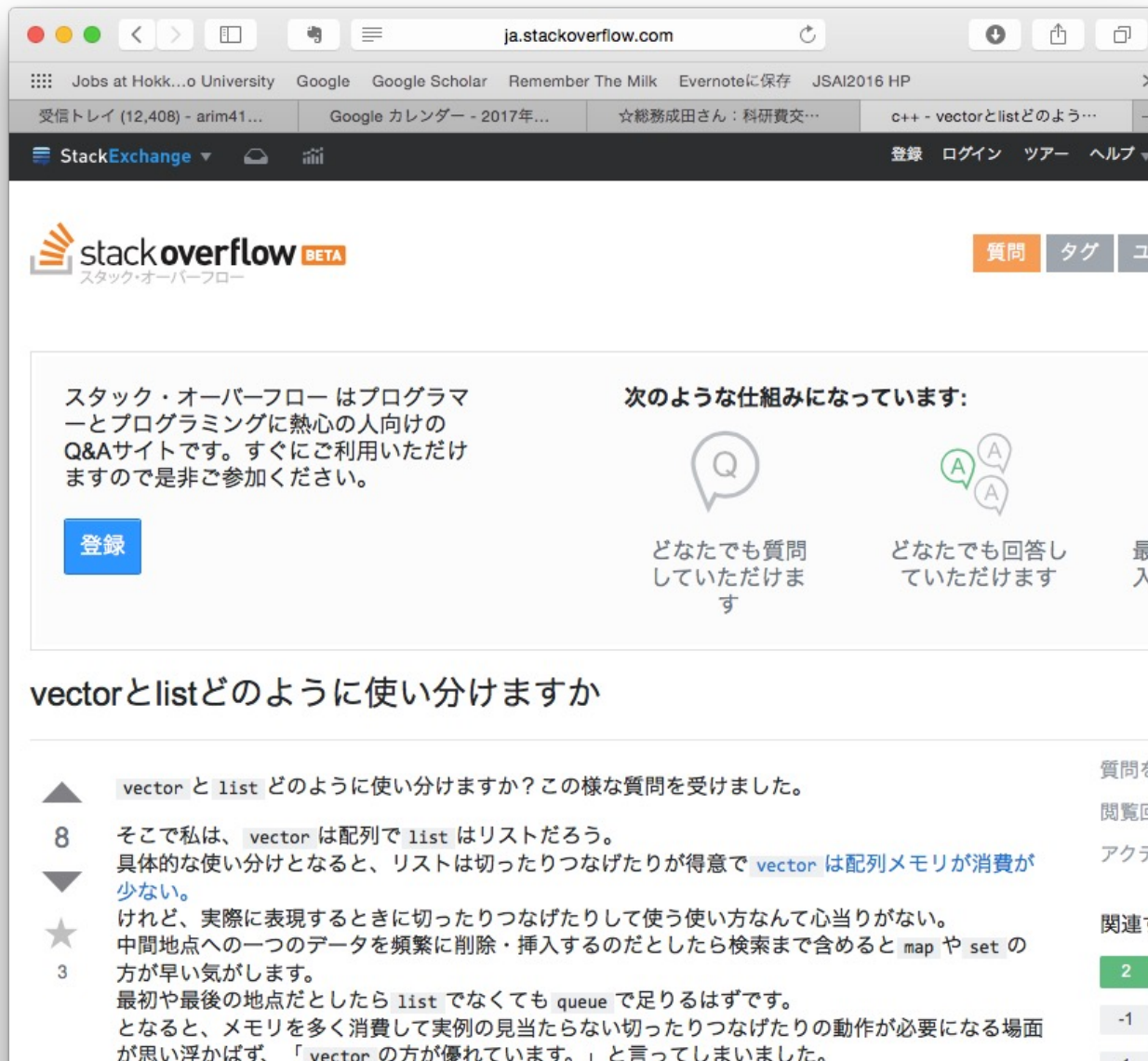


- **もっと複雑なプログラム**が書けるようになる

- **プログラム能力があがる!**

大事なところに
集中できる

- データ構造がわかると, こういう質問にも答えられます



□ Q「vectorとlistはどのように使い分ければよいですか？」

□ Q「List と, Stack とQueueはどちらがうのですか？」

データ構造とアルゴリズムの定番教科書

- ◆ この授業の教科書（一冊持っていると良い。自習には本は必須。）
 - 茨木俊秀, Cによるアルゴリズムとデータ構造, 照晃堂, 2916円
 - 平田富夫, アルゴリズムとデータ構造<改訂 C言語版>, 森北出版, 2376円.
- ◆ Aho, Hopcroft, Ullman
 - The Design and Analysis of Computer Algorithms, Addison Wesley, 1974. (最初のデータ構造の教科書の一つ)
- ◆ Knuth
 - The Art of Computer Programming, Addison-Wesley
1巻(基本算法), 2巻(準数値算法), 3巻(整列と探索), 1975
 - 基本部分の百科辞典. 機械語で記述. 全部読んだ人は少ない?
- ◆ Cormen, Leiserson, Rivest, and Stein
 - Introduction to Algorithms, 3rd Edition, MIT Press, 2009.
 - 現在, 最も定評がある教科書. 企業やプロコンなどでの勉強会の定番. 日本語訳有, 3巻組. (興味があったら図書館で借りて読んでみよう)

今日のアルゴリズム

ユークリッドの互助法

「情報理工学演習III」を履修している人へ：
初回の演習の課題は、ユークリッドの互除法です。

ユークリッド

ユークリッドの「原論」の
1570年発行の英語版
初版本の表紙
The frontispiece of Sir
Henry Billingsley's first
English version of Euclid's
Elements, 1570



- Euclid (Greek: Εὐκλείδης)
- 紀元前300 生まれ
- アレキサンドリアの数学者.
- ユークリッドの「幾何学原論」(Elements)は, 歴史に残る数学の名著
- 後世に大きな影響を与えた

■ 「原論」13巻の内容

- ✓ 1巻: 平面図形の性質
- ✓ 2巻: 面積の変形 (幾何的代数)
- ✓ 3巻: 円の性質
- ✓ 4巻: 円に内接・外接する多角形
- ✓ 5巻: 比例論
- ✓ 6巻: 比例論の図形への応用
- ✓ 7巻: 数論
- ✓ 8巻: 数論
- ✓ 9巻: 数論
- ✓ 10巻: 無理量論
- ✓ 11巻: 立体図形
- ✓ 12巻: 面積・体積
- ✓ 13巻: 正多面体

エジプト中部のオクシュリユンコス
で発見された『原論』の断片
(第2巻の命題5)。紀元100年頃(
<http://en.wikipedia.org/wiki/Euclid>)



ちょっと準備：最大公約数問題 (GCD)

- 入力: 2個の正整数 a, b
- 出力: a と b の最大公約数を答えよ

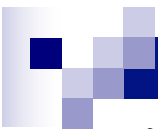
最大公約数問題とは？

最大公約数(greatest common divisor, GCD)

正整数を $a, b, c \geq 0$ とする

- c が a の約数(divisor)であるとは:
 - c が a を割り切ること, c で a を割った余りが0であること.
- c が a と b の公約数(common divisor)であるとは:
 - c が a の約数, かつ, c が b の約数であること.
 - 性質: 数 l は任意の a と b の公約数
- c が a と b の最大公約数(GCD)であるとは:
 - c が, a と b の公約数の中で最大のものであること.

性質: 正整数 a と b の最大公約数は, ちょうど一つ存在する



練習（最大公約数）

- 問題1) 24と36の最大公約数を求めてみよう!
- 問題2) 最大公約数を求める手順（アルゴリズム）を書いてみよう.

GCDの素朴なアルゴリズム

アルゴリズムの擬似コードによる表現

入力: 非負整数 $a_0, a_1 \geq 0$

```
 $n \leftarrow a_1$ 
```

```
while( true ) {
```

```
  if  $a_0$ と $a_1$ が共に $n$ で割り切れる then
```

```
    return  $n$ 
```

```
  end if
```

```
   $n \leftarrow n - 1$ 
```

```
} //while文のおわり
```

無限に繰り返す

読みやすい、

n の値を返してこの
手続きから抜ける

メモ: この素朴なアルゴリズムは, 正しくGCDを求める

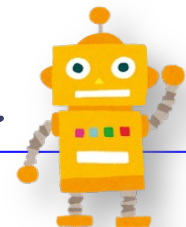
→ ただし, 計算効率があまり良くない

→ 追加勉強: while文を何回実行するか, 確かめてみよう).

高速なアルゴリズム： ユークリッドの互除法

- 知られている最古のアルゴリズム
 - 2000年くらい前（紀元前200年くらいだから）
- 二つの数の最大公約数をみつけるアルゴリズム
 - どんな入力に対しても、必ず停止して、正しい答えを出す。
 - 例) 24と36の最大公約数を求めよ

速いだけでなく、「エレガント」
(優雅)なアルゴリズムです



ユークリッドの互除法 (高速なアルゴリズム)

- 入力:二つの整数 X と Y
- 出力: X と Y の最大公約数 Z を求める
- 手順:
 1. $Z := 1$ とおく.
 2. 除数 $P = 2, 3, 4, 5, \dots, \min(X, Y)$ に対して, 順に X と Y の両方を割り切るかどうか, 試す.
 3. そのような P が見つかったら, 最初に成功した数を P として, $Z := Z \times P$ と置き換える. 上記を P が見つかる限り, 繰り返す
 4. もしそのような P がなかったら, 答えとして Z を出力して停止する.

ユークリッドの互除法（高速なアルゴリズム）

ループを用いた実装方法
擬似コードによる表現

```
 $a \leftarrow a_0, b \leftarrow a_1$   
while ( true ) {  
     $r \leftarrow a$  を  $b$  で割った余り  
    if  $r == 0$  then  
        return  $b$   
    end if  
     $a \leftarrow b, b \leftarrow r$   
} //くりかえしの終わり
```

- 補足: 繰り返し文(ループ)は, 本体の命令文をずっと繰り返す.
- C言語なら `true` の代わりに `1` と書くこと.
- 途中の条件文でループを脱出する.

書き方2: ユークリッドの互除法の擬似コード

- 最大公約数を求めるユークリッドの互除法は、再帰手続きを用いて書くことができる。
- 簡潔で分かりやすい。

```
int gcd(x,y) {  
    if (x > y) then gcd(y,x);  
    else if x = 0 then y;  
    else gcd(mod(y,x), x);  
}
```

再帰を用いた実装方法

擬似コードによる表現

メモ: 実は、ループで書いても、再帰で書いても計算量は同じです。(計算量の正確な定義は次回)

第1回ガイダンス

■ 今日の内容

- アルゴリズムとデータ構造とは何か
- 今日のアルゴリズム: 最大公約数問題を解く「ユークリッドの互除法」を学ぼう

■ ポイント

- アルゴリズムとデータ構造の定義
- 「ユークリッドの互除法」を理解すること
(情報理工学演習IIIを取っている人は初回の課題)